To avoid this problem, we start by initializing all the actions for the agents randomly, and then we can compute the joint actions using these random actions. But if that's all we did, using joint actions wouldn't be much help, since they're random. In the pseudocode in listing 9.2 we address the problem by rerunning this process a few times (that's the `for m in range(M)` part, where `M` is some small number like 5). The first time we run this, the joint action will be random, but then all the agents will have taken an action based on their Q functions, so the second time it will be slightly less random, and if we keep doing this a few more times, the initial randomness will be sufficiently diluted and we can take the actions at the end of this iteration in the real environment.

Listing 9.2. Pseudocode for neighborhood Q-learning, part 2

```
# Initialize actions for all agents

for m in range(M):                          1
    for j in agents:
        state = environment.get_state()
        neighbors = get_neighbors(j, num=3)
        joint_actions = get_joint_action(neighbors)
        q_values = Q(state, joint_actions)
        j.action = policy(q_values)

for j in agents:                           2
    environment.take_action(j.action)
    reward = environment.get_reward()
```

- ***1*** Iterates through the process of computing joint actions and Q values a few times to dilute the initial randomness
- ***2*** Needs to loop through agents again to take the final actions that were computed in the previous loop

Listings 9.1 and 9.2 show the basic structure of how we will implement neighborhood Q-learning, but one detail we've left out is exactly how to construct the joint-action space for the neighboring agents. We build a joint action from a set of individual actions by using the *outer product* operation from linear algebra. The simplest way to express this is to "promote" an ordinary vector to a matrix. For example, we have a length 4 vector and we could promote it to a 4×1 matrix. In PyTorch and numpy we can do this using the `reshape` method on a tensor, e.g.,